Microsoft Sentinel Incident Automation

Disabled-account sign-in attempts: automated triage and SOC notification with a Logic App playbook

Item	Detail
Prepared by	Mukhtar
Date	October 11, 2026
Target role	Security Infrastructure Engineer, Microsoft Sentinel (SOC)
Environment	Sentinel workspace law-sec-central, resource group rg-infra-sec (East US)
Build method	Azure portal or Terraform (Both supported)

1. Executive summary

The SOC was opening every disabled-account sign-in incident by hand, copying details into email and tagging them inconsistently. This ticket automates first-response triage with Microsoft Sentinel, end to end, while keeping the automation strictly to notification and triage: nothing in it changes a user, a role or a device.

A scheduled analytics rule detects failed sign-ins to disabled accounts (Entra error 50057) and raises one incident per account with Account and IP entities. Two automation rules then run in a fixed order: the first sets the incident to Active and tags it auto-triaged, the second runs a Logic App playbook. The playbook, using a system-assigned managed identity, writes a comment on the incident and emails the SOC.

Key results

- A disabled test user was used to generate real 50057 sign-in failures; they appeared in Entra sign-in logs and in the Sentinel SigninLogs table.
- Incident #3 was created automatically (Medium, tactics Initial Access and Credential Access), tagged auto-triaged, set to Active and commented on by the playbook.
- The notification email arrived with the incident title, account, severity and a deep link; every playbook run in the run history shows Succeeded.
- Permissions follow least privilege: Microsoft Sentinel Responder for the playbook identity and Microsoft Sentinel Automation Contributor for the Azure Security Insights service, both on rg-infra-sec only.

2. Requirements

The SOC lead asked for consistent tagging, an incident comment with context and a notification to the SOC mailbox, with no automatic changes to identities. The trigger chosen for this build is "trying to log in with a disabled account".

Requirement	Delivered
Detection rule for disabled-account sign-ins	Template rule "Attempts to sign in to disabled accounts" (Microsoft Entra ID content), query on ResultType 50057 across SigninLogs and AADNonInteractiveUserSignInLogs
Playbook with Sentinel incident trigger	pb-sentinel-notify-disabledacct (Logic App, Consumption): Get Accounts, For each account add a comment and send an email
Managed identity, least privilege	System-assigned identity with Sentinel Responder on rg-infra-sec; Automation Contributor for Azure Security Insights
Automation rule 1: triage (order 1)	ar-disabledacct-triage: incident created, status Active, tag auto-triaged
Automation rule 2: notify (order 2)	ar-disabledacct-notify: incident created, run the playbook
No destructive or auto-close actions	Automation limited to status Active, a tag, a comment and an email
End-to-end test	Disabled test user, repeated sign-in attempts, incident and email verified

3. Solution architecture

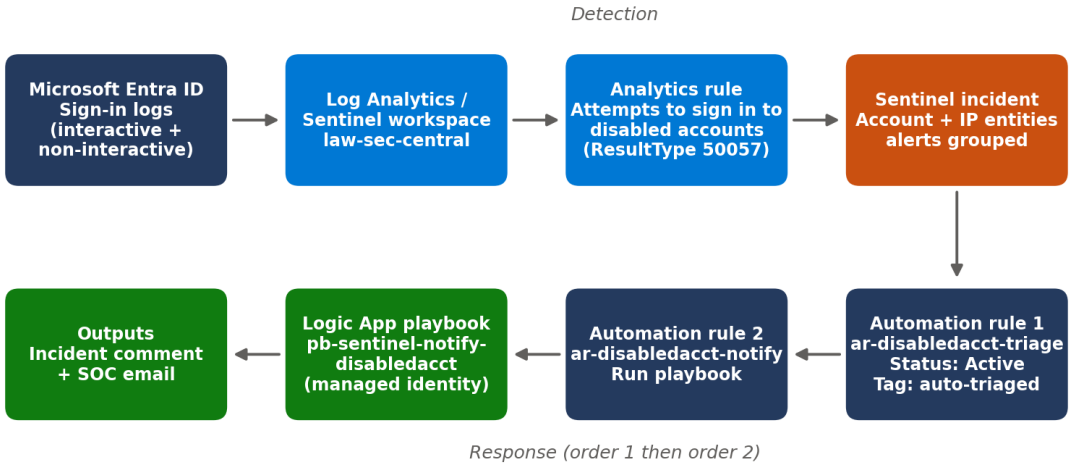


Figure 1. Detection and response flow for the disabled-account scenario

Entra ID sign-in logs flow into the Log Analytics workspace behind Sentinel. The scheduled rule turns matching sign-ins into alerts and groups them into incidents. When an incident is created, the automation rules fire in order: triage first so the incident is already tagged and active, then the playbook, which enriches the incident with a comment and notifies the SOC.

4. Implementation walkthrough (Azure portal)

Step 1. Confirm sign-in telemetry reaches Sentinel

The Microsoft Entra ID data connector is Connected, with the last log received two minutes before the capture. The data types received include SigninLogs, AuditLogs and AADNonInteractiveUserSignInLogs, so both interactive and non-interactive sign-ins are available to the rule.

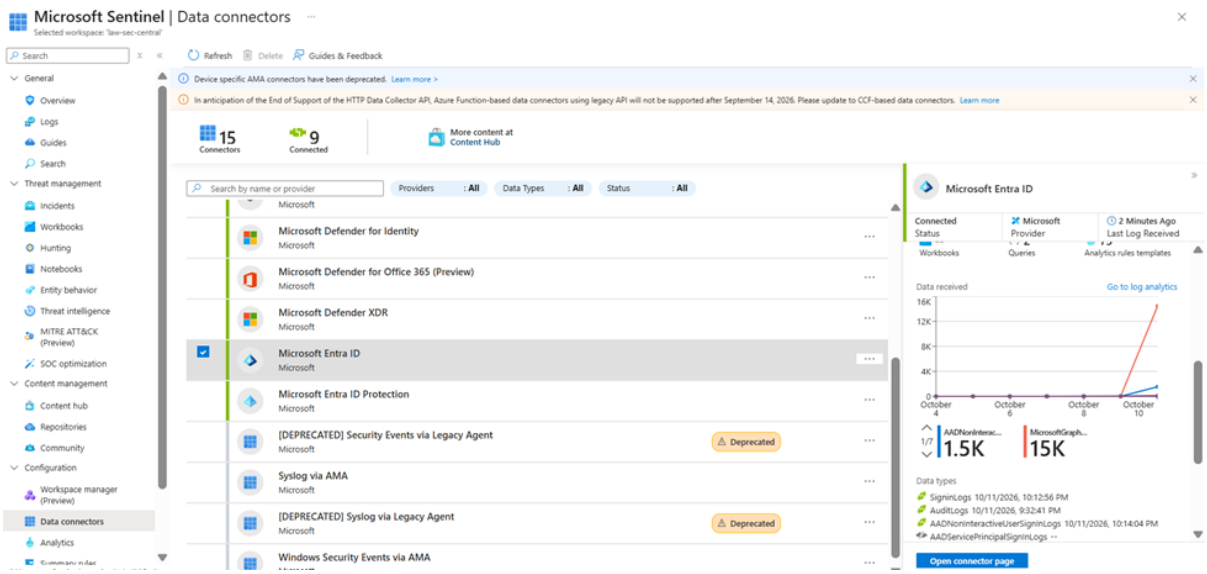


Figure 2. Microsoft Entra ID connector: Connected, with SigninLogs and AADNonInteractiveUserSignInLogs received

What: Verify that the raw material for the detection exists in the workspace.

Why: A rule on an empty table never fires, and a missing connector is the most common reason a detection appears broken.

Who: The Security Infrastructure Engineer owns connector health; the Entra administrator owns the diagnostic export and licensing.

Step 2. Create and disable a test user

A dedicated test account was used as a safe stand-in for a leaver or a stale service account, then disabled. Sign-in attempts were made with the disabled account.

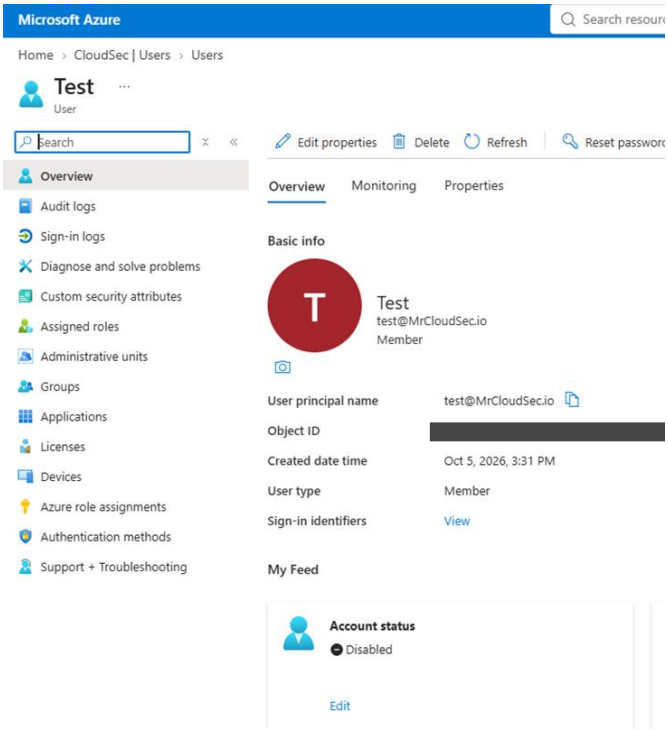


Figure 3. Test user with Account status: Disabled

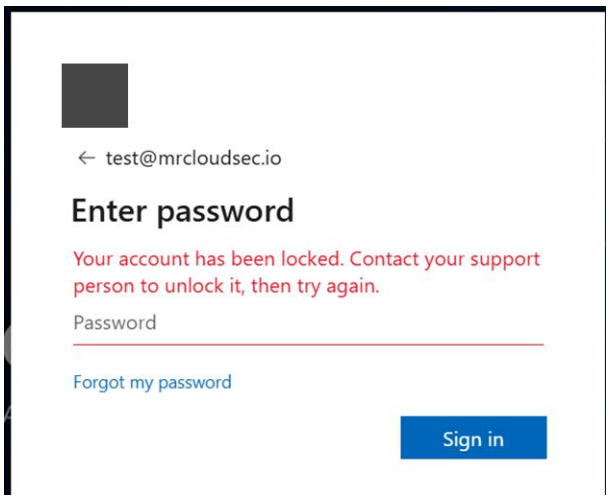


Figure 4. Sign-in page response for the disabled account

The sign-in page shows "Your account has been locked", which is the friendly user-facing message for a disabled account. It is not smart lockout (error 50053). The logs below confirm the real reason: ResultType 50057, "The user account is disabled."

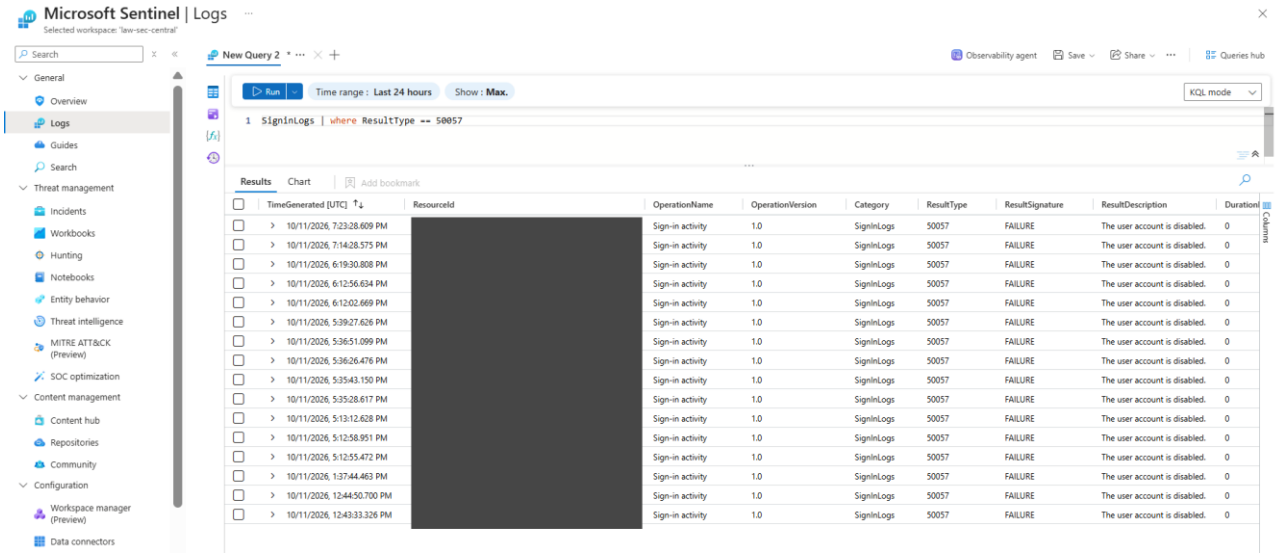


Figure 5. Sentinel Logs: SigninLogs where ResultType == 50057 (tenant identifiers redacted)

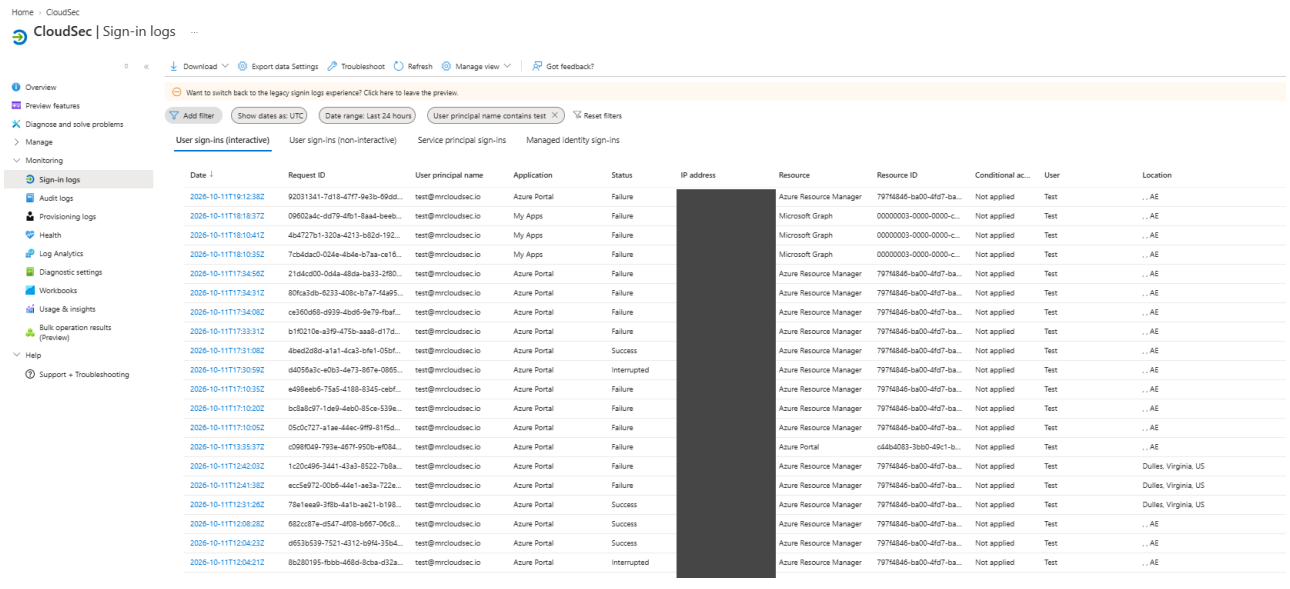


Figure 6. Entra ID sign-in logs, interactive: repeated failures for the test user (IP addresses redacted)

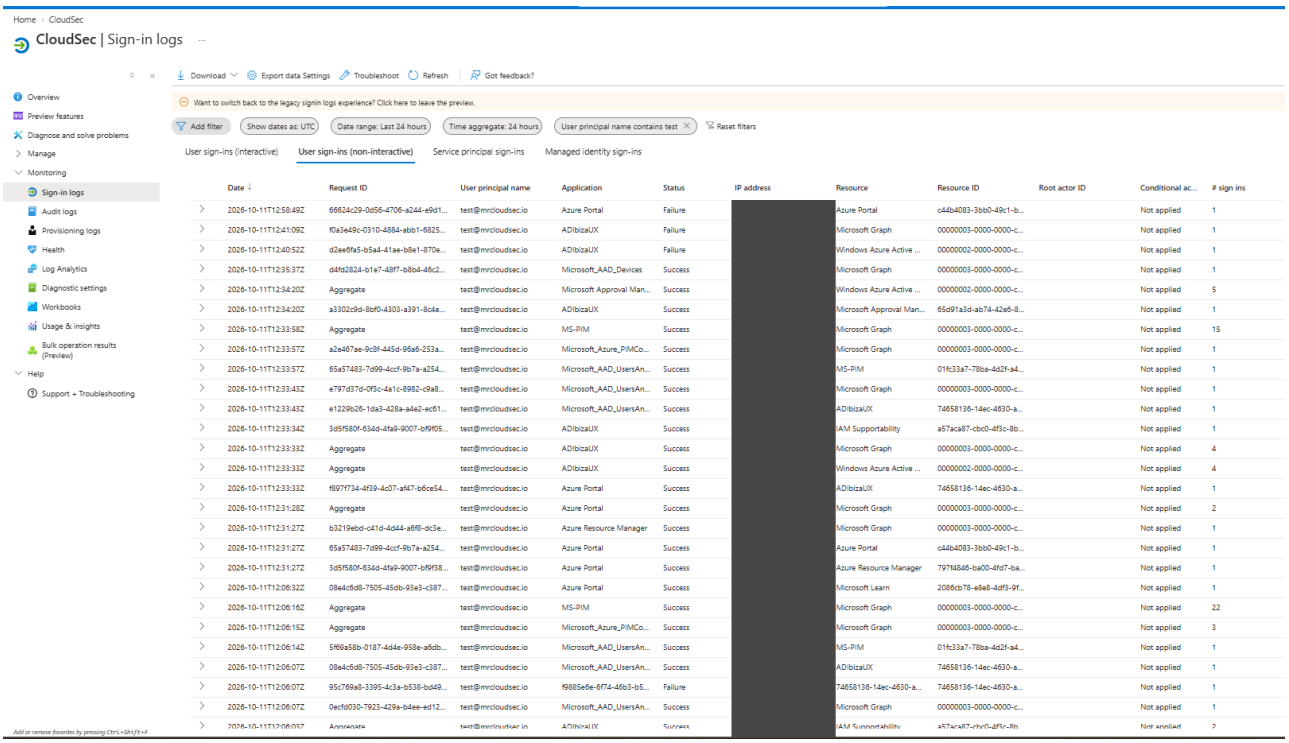


Figure 7. Entra ID sign-in logs, non-interactive (IP addresses redacted)

> **What:** Generate controlled, realistic 50057 events using an account created only for testing.
>
> **Why:** Real leavers, stale service accounts and attackers replaying old credentials all produce this signal; testing with a real user would be unsafe.
>
> **Who:** HR and IAM own the disable process; the SOC only watches for attempts afterwards.

Step 3. Configure the analytics rule

The rule is the Microsoft template "Attempts to sign in to disabled accounts" from the Content hub: severity Medium, tactics Initial Access and Credential Access, technique T1078, source Microsoft Entra ID. The query filters ResultType 50057 and summarizes by user, IP and log type across both sign-in tables. The template alerts when a user fails against a number of distinct applications that meets the threshold variable; the template default is 3, and it was set to 1 for the lab so a single test fires the rule.

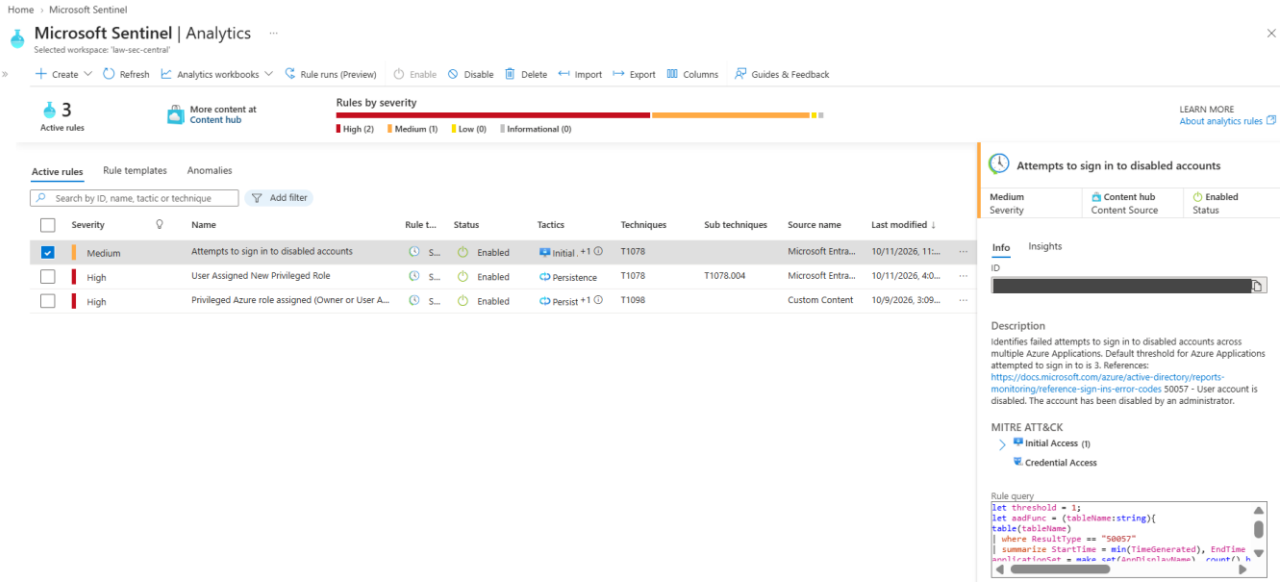


Figure 8. Active analytics rules: the disabled-accounts rule is enabled; query shows threshold = 1

Setting	Value used
Query frequency	Every 5 minutes
Lookback	Last 5 hours
Alert threshold	Number of results greater than 0
Event grouping	Group all events into a single alert
Suppression	Off
Entity mapping	Account (FullName = UserPrincipalName, Name, UPNSuffix); IP (Address = IPAddress)
Threshold variable	1 in the lab (template default 3)

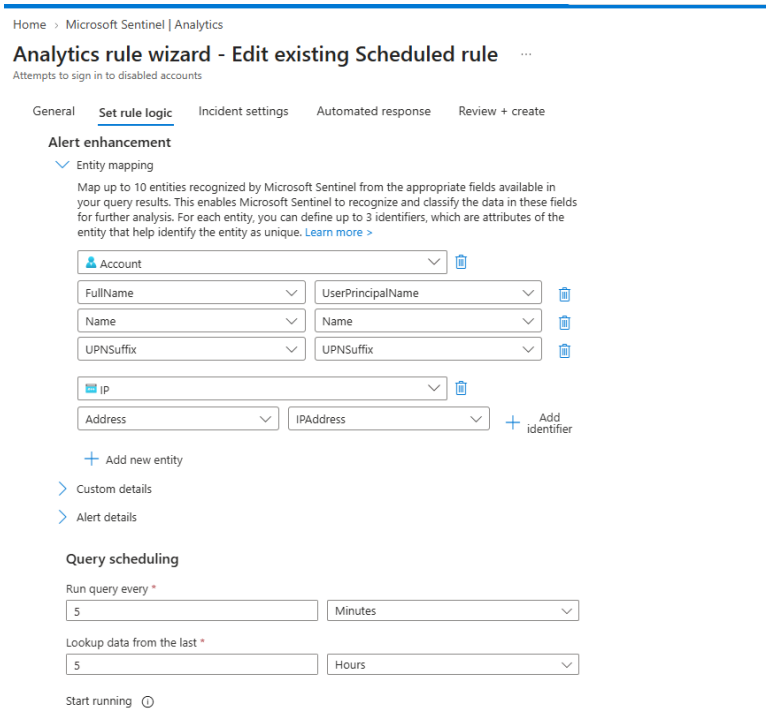


Figure 9. Entity mapping and query scheduling

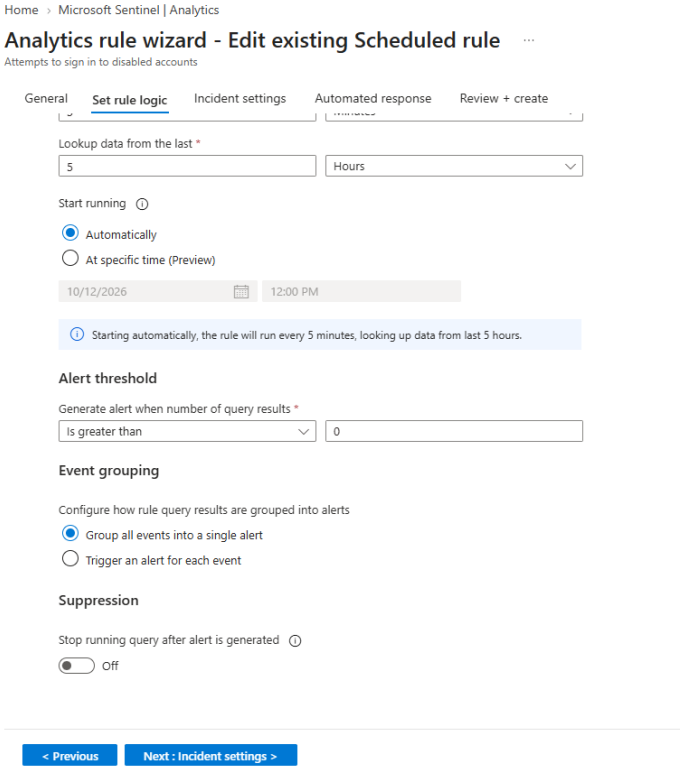


Figure 10. Alert threshold, event grouping and suppression settings

What: Turn matching sign-ins into alerts with Account and IP entities and group them into incidents.

Why: Entity mapping lets analysts and the playbook pivot to the user and IP, and the playbook reads the account from the entities. Grouping keeps repeated attempts from creating many incidents and many emails.

Who: The detection engineer owns the rule; on a production Financial Services workspace a change like this would need a peer approval first.

Step 4. Create the playbook with a managed identity

The Logic App pb-sentinel-notify-disabledacct was created in rg-infra-sec with a system-assigned managed identity, so the playbook has no stored credentials or secrets to rotate.

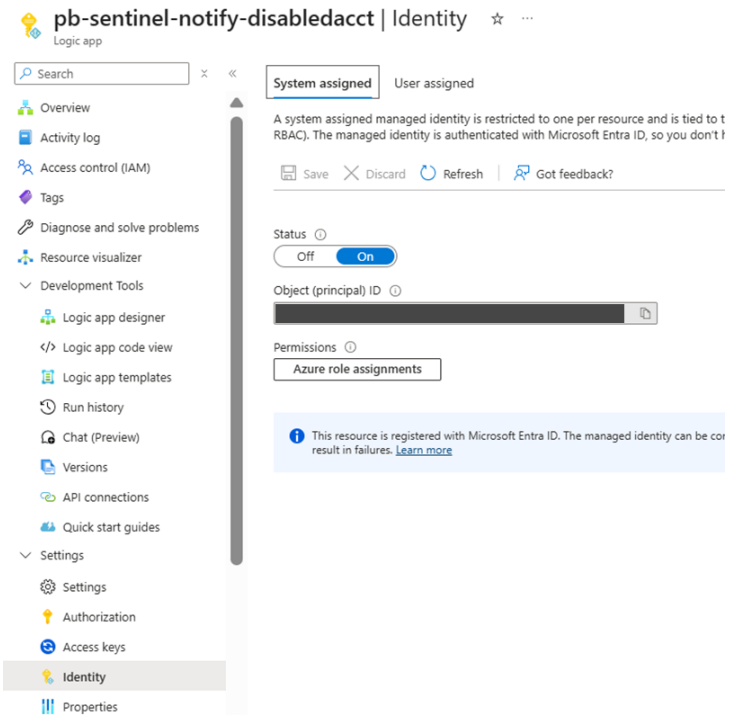


Figure 11. Logic App Identity: System assigned status On

Step 5. Assign least-privilege roles

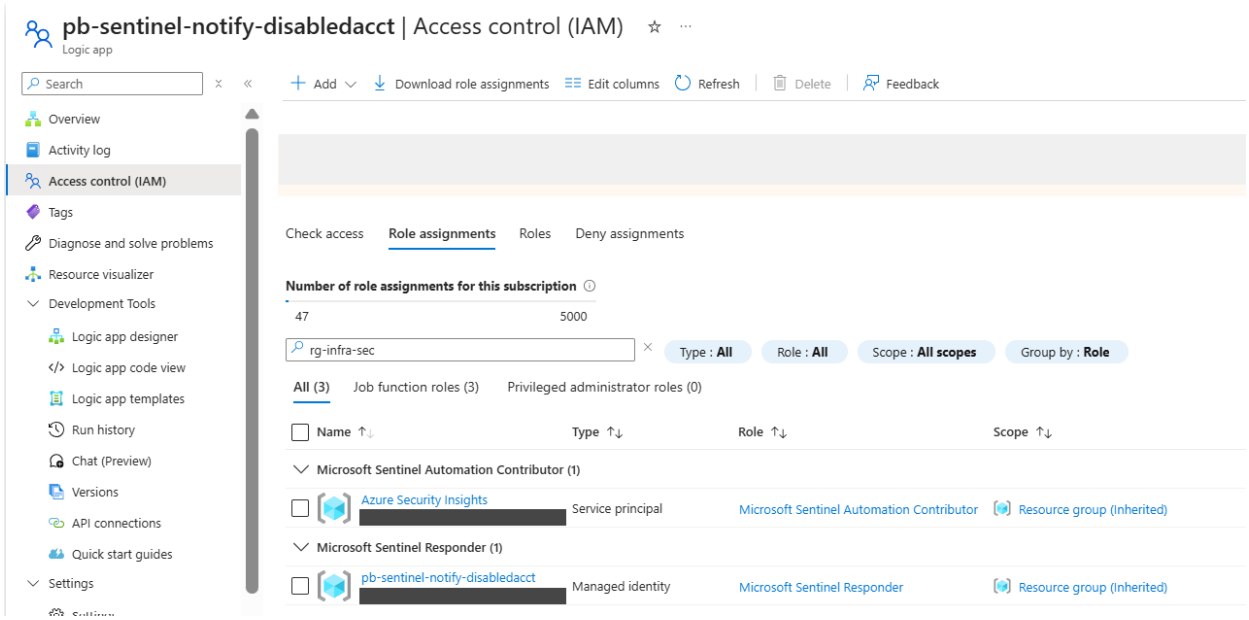


Figure 12. Role assignments at rg-infra-sec: Automation Contributor (Azure Security Insights) and Responder (playbook identity)

What: Two role assignments serve two different purposes, both scoped to the resource group.

Why: Sentinel Responder lets the playbook read incidents and write comments. Automation Contributor lets the Sentinel service itself run playbooks in this resource group; without it the playbook is not selectable in an automation rule.

Who: The cloud security engineer assigns them; an Owner or User Access Administrator approves.

Step 6. Build the workflow in the Logic App designer

The workflow is: Microsoft Sentinel incident trigger, Entities - Get Accounts, then a For each loop over the accounts containing Add comment to incident (V3) and Send an email (V2). The Sentinel connection uses the managed identity (connection sentinel-mi).

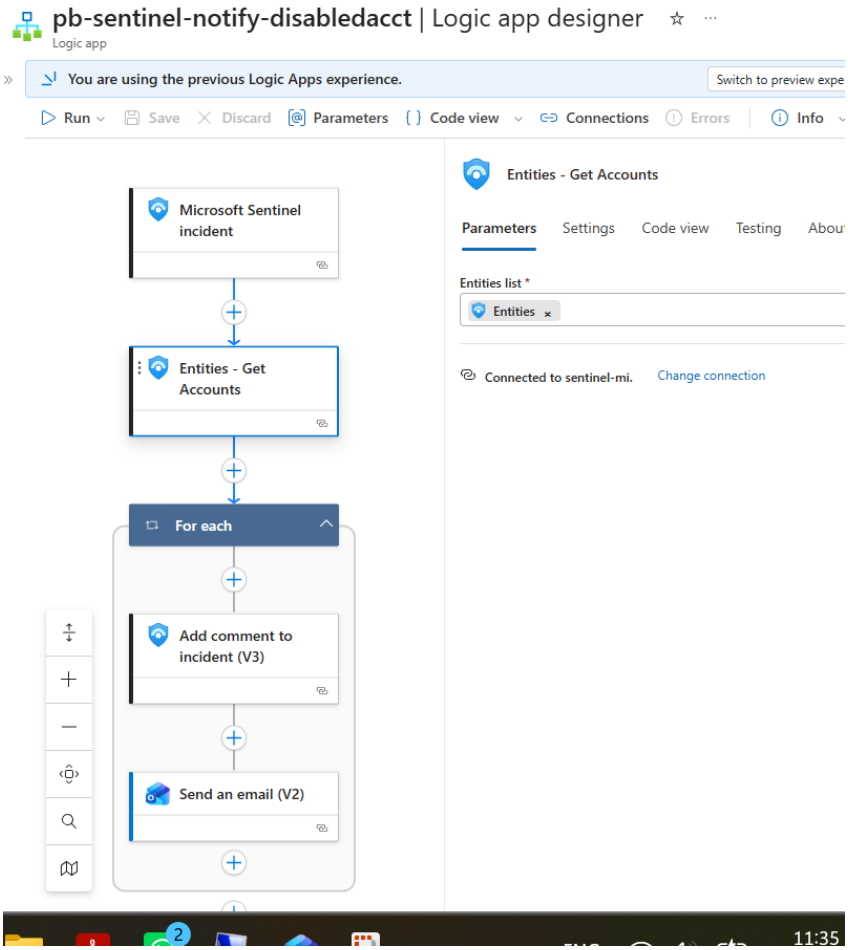


Figure 13. Designer: trigger, Entities - Get Accounts, For each

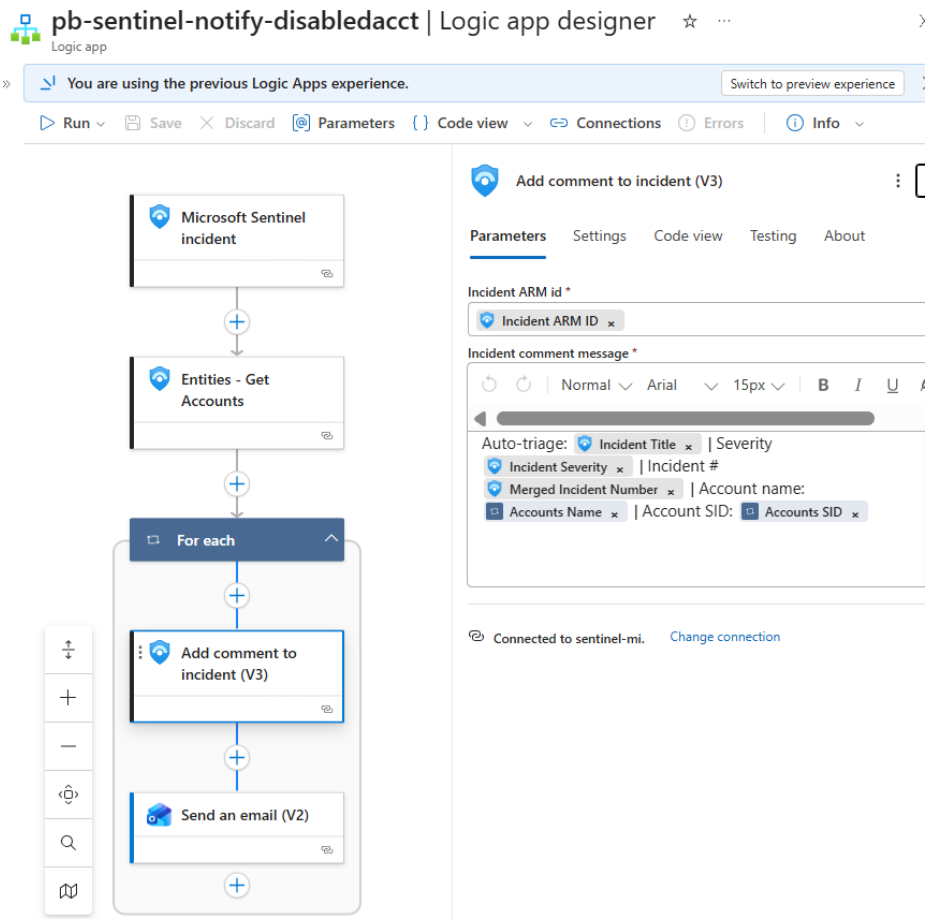


Figure 14. Add comment to incident (V3): title, severity, account name and SID

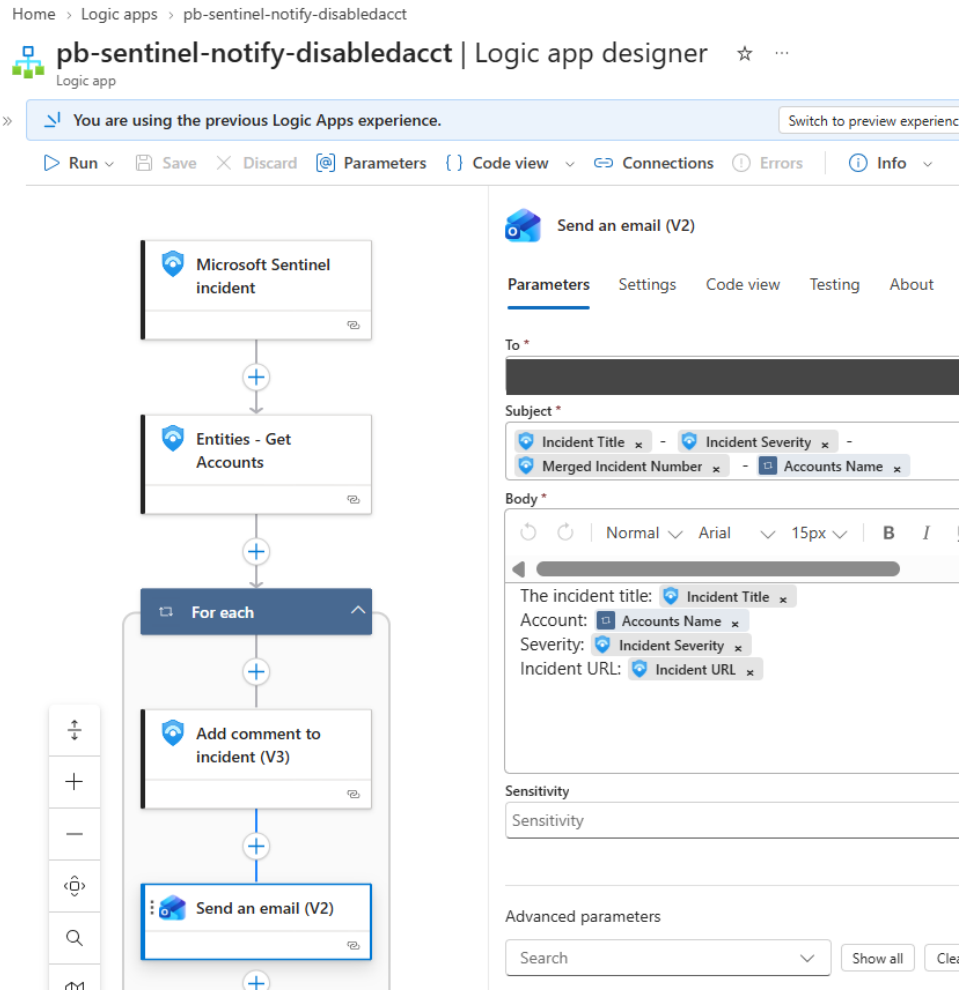


Figure 15. Send an email (V2): subject and body with incident title, account, severity and URL (recipients redacted)

What: Add an audit trail inside the incident and notify the SOC by email.

Why: The comment survives even if the email is missed or filtered, and the account details come from the mapped entities rather than free text.

Who: The playbook author owns it and the SOC lead approves what it may do. It only comments and emails; it never touches an identity.

Step 7. Create the automation rules

Two rules apply to incidents created by the disabled-accounts analytics rule. Order 1 (ar-disabledacct-triage) changes the status and adds the tag; order 2 (ar-disabledacct-notify) runs the playbook. Both are enabled with no expiration.

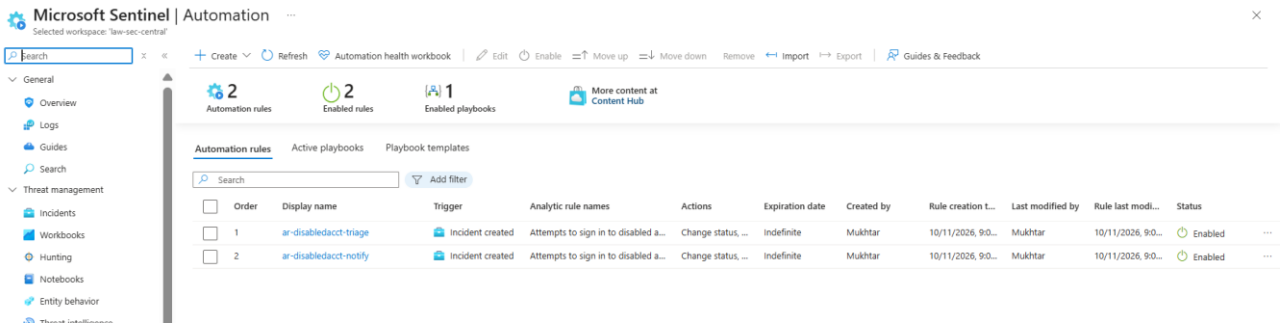


Figure 16. Automation rules: triage (order 1) and notify (order 2), both triggered on incident creation

What: Two rules with a defined order, scoped by analytics rule name.

Why: Triage runs first so the playbook sees a tagged, active incident. Scoping by rule name keeps the automation off incidents from other detections. No rule closes incidents, because wrong automation can silently hide a real attack.

Who: The SOC owns automation rules and reviews them as part of content change control.

Step 8. Test and verify results

After the sign-in attempts, incident #3 appeared with the title "Attempts to sign in to disabled accounts involving one user", severity Medium, status Active, tag auto-triaged and the playbook comment.

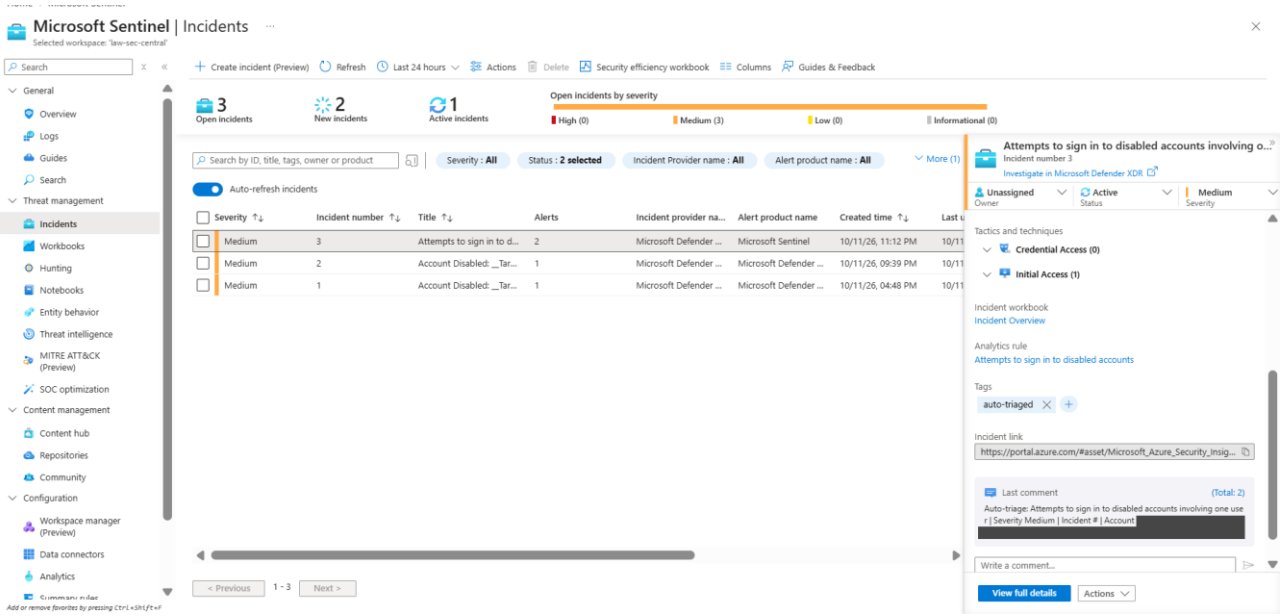


Figure 17. Incident #3: Active, tag auto-triaged, analytics rule name, playbook comment (SID redacted)

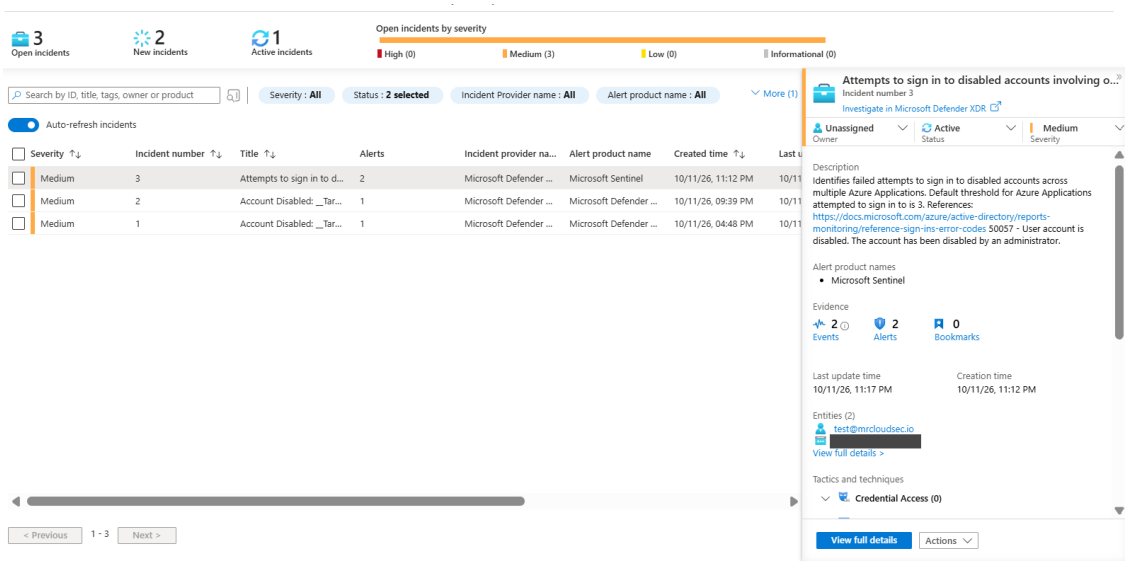


Figure 18. Incident #3 details: description, evidence, mapped Account entity (IP redacted)

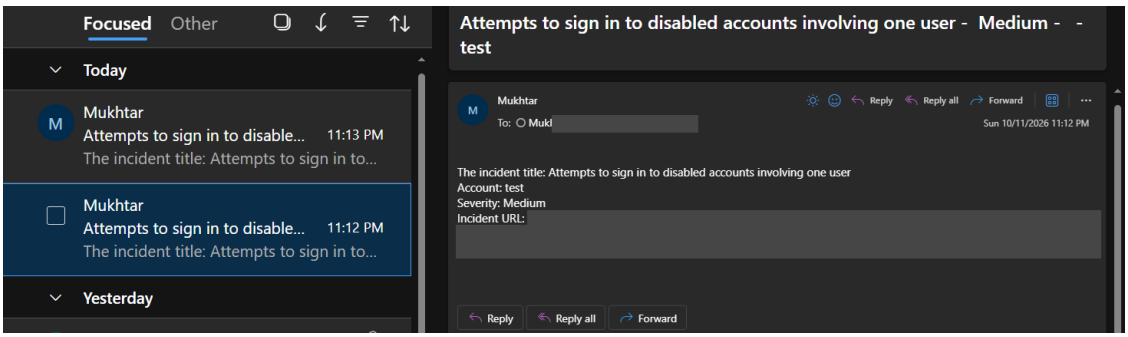


Figure 19. Notification email delivered by the playbook (recipients and URL redacted)

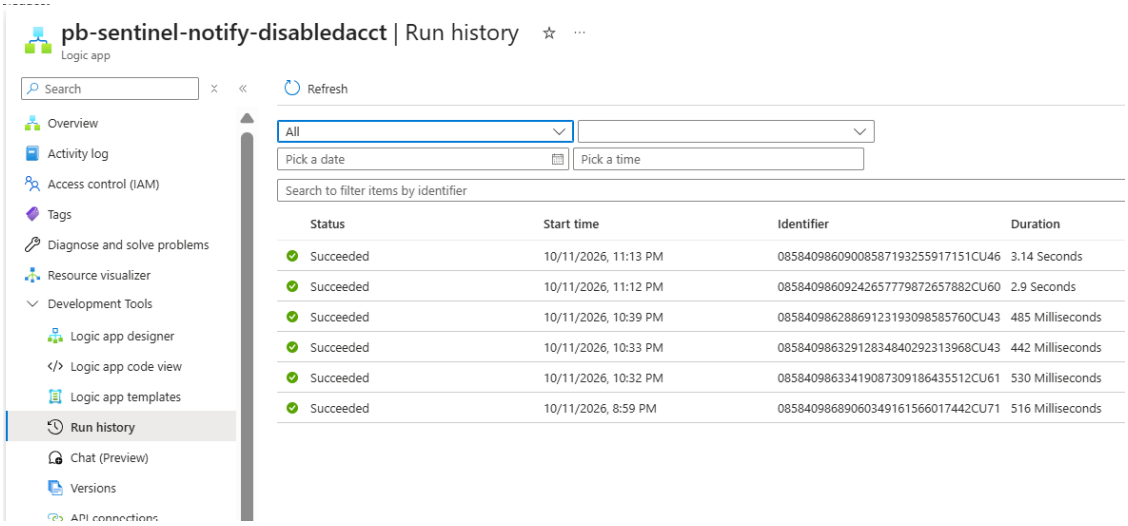


Figure 20. Logic App run history: all runs Succeeded

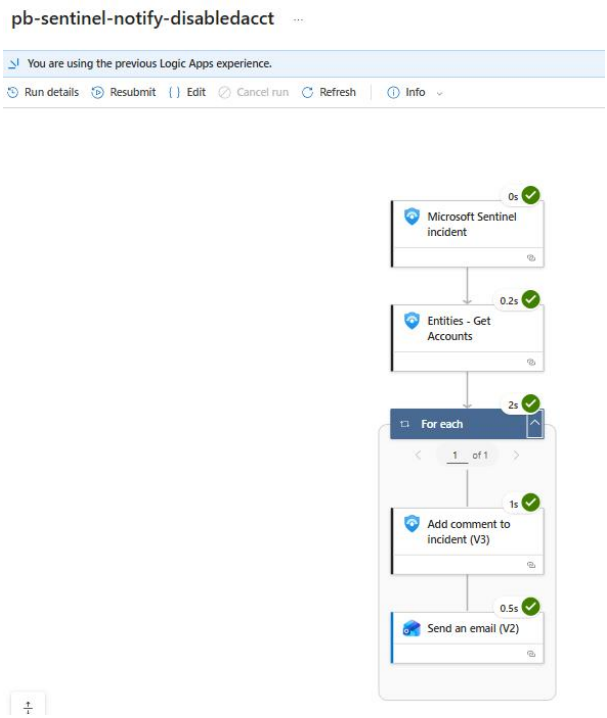


Figure 21. A playbook run: every step succeeded; For each ran once (one account)

Scoping check: the two Defender XDR incidents created when the account was disabled were not tagged and had no playbook comment, so the automation applies only to the disabled-account sign-in rule.

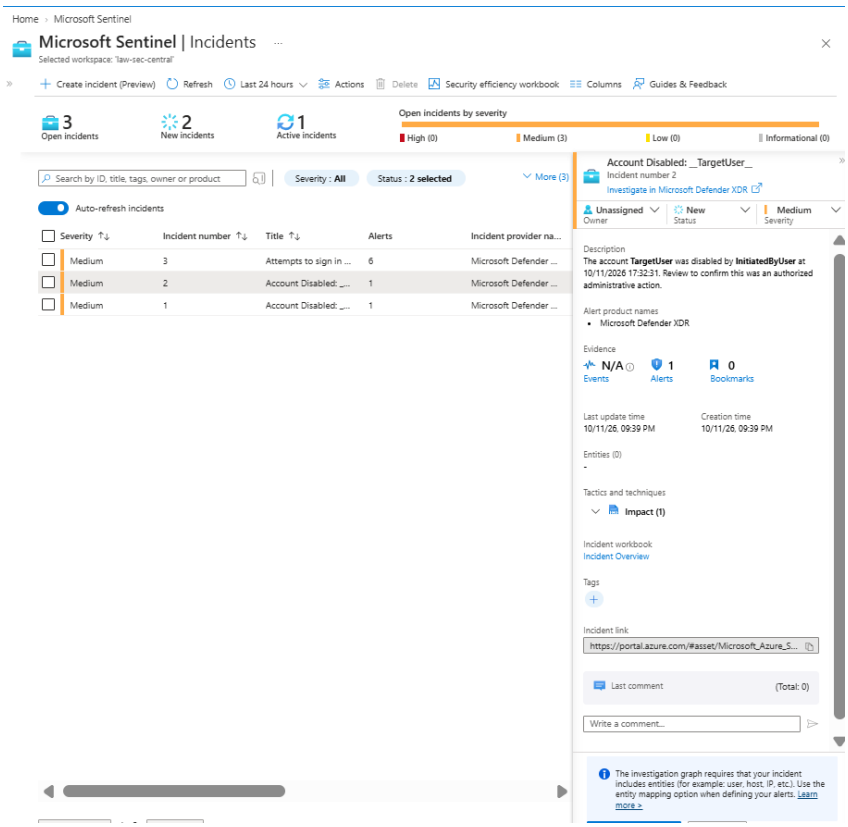


Figure 22. Incident #2 (Defender XDR, account disabled): no tags, no comments

5. Validation against acceptance criteria

Criterion	Status	Evidence
Incident created from disabled-account sign-in attempts	**Pass**	Incident #3, Medium, Account and IP entities
Incident tagged auto-triaged and set to Active	**Pass**	Incident panel in Figure 17
Playbook comment added to the incident	**Pass**	Last comment shown, 2 comments in total
Notification email delivered	**Pass**	Email with title, account, severity, link
Playbook runs succeed	**Pass**	Run history and run diagram
Managed identity with least privilege	**Pass**	Responder plus Automation Contributor on rg-infra-sec only
Rule order: triage before notify	**Pass**	Orders 1 and 2 in the automation rules list
No automatic closure or identity changes	**Pass (by design)**	Actions limited to status, tag, comment, email

6. Observations and tuning notes

- **Incident number was blank.** The comment and email used the Merged Incident Number token, which is only filled for merged incidents. The fix is to use the Incident number token from the trigger.
- **Alert count grew to 6 in one incident.** The rule runs every 5 minutes with a 5-hour lookback, so the same events are re-read. Grouping by account kept this to a single incident. For production, set the

lookback close to the schedule (for example 1 hour to allow for ingestion delay) and use suppression if repeats continue.

- **Two emails one minute apart.** Run history shows runs at 11:12 PM and 11:13 PM. This is consistent with the overlap above; confirm that the notify rule triggers on incident creation only.
- **Threshold.** The lab uses a threshold of 1 distinct application. The template default of 3 is a better production starting point, because one stray retry from a leaver should not page the SOC.
- **Lab shortcuts.** Recipients are hard-coded and the Outlook connection uses a personal sign-in. Production would use a shared SOC mailbox or a Teams webhook so notifications survive staff changes.
- **Loop design.** The email sits inside the For each loop, so an incident with several accounts would send several emails. In production, loop for the comment and send one email listing all accounts.
- **Different detections.** Incidents #1 and #2 (account disabled) come from Defender XDR and mark the moment of disabling; the automated incident marks later attempts against the disabled account. Together they form a useful timeline.
- **Cost.** The connector chart shows a large volume of Microsoft Graph activity records. Ingest only tables that have detections and consider data collection rule filtering or lower-cost tiers.

7. Risk register

Risk	Impact	Likelihood	Mitigation
Alert fatigue from repeated sign-in attempts	Medium	High	Group alerts by account, raise the threshold, align lookback with schedule, suppress after alert, exclude known service accounts with a watchlist
Automation hides a real attack	High	Low	No auto-close; automation limited to tagging, commenting and notifying; review automation rules under change control
Playbook identity over-privileged	High	Low	Responder only, scoped to rg-infra-sec; no Owner or Contributor; review role assignments periodically
Playbook cannot run or comment (403)	Medium	Medium	Keep Automation Contributor for Azure Security Insights; allow for role propagation; monitor run failures
Notification breaks when a person leaves	Medium	Medium	Shared SOC mailbox or Teams webhook; parameterize recipients
Detection blind spot if the connector or a log type is off	High	Medium	Monitor connector health and ingestion; keep both interactive and non-interactive logs enabled
Wrong or empty fields in notifications	Low	Medium	Test playbook output with a manual run on a real incident; use the correct dynamic content
Log ingestion cost growth	Medium	Medium	Ingest only needed tables; use data collection rule filtering and appropriate tiers

8. Lessons learned and next steps

- Order matters: automation rules run in sequence, so put property changes before actions that depend on them.

- Permissions have two sides: the playbook identity needs Responder, and Sentinel itself needs Automation Contributor on the playbook resource group.
- A good detection is a balance: threshold, lookback and grouping decide whether the SOC sees one clear incident or noise.